Manual Técnico — 3xtrat3rr3str3D

Compilador de tres lenguajes de alto nivel (Lat / Pig Latin, Y? y Zetariano) hacia código de tres direcciones (C3D) y, finalmente, ensamblador RISC-V ejecutable en RARS.

Dato	**Descripción**
Proyecto	Compi2 — Proyecto 2: traducción a ensamblador y extensiones de Z
Artefacto	com.navi:3xtrat3rr3str3D:1.0-SNAPSHOT
Lenguaje de implementación	Java 17
Herramienta de análisis	ANTLR 4.13.2
Construcción	Maven 3.9.x
Plataforma destino	RISC-V de 32 bits, simulador RARS 1.6
Fecha de documentación	2026

1. Introducción

1.1 Objetivo

Construir un compilador que reciba programas escritos en los tres lenguajes de la Resistencia y permita:

1. Validar el código mediante análisis léxico, sintáctico y semántico.
2. Generar cuartetas de código de tres direcciones.
3. Traducir las cuartetas a ensamblador RISC-V, aplicando los conceptos de stack, heap, marcos de activación y llamadas.
4. Ejecutar el resultado en RARS, con entrada y salida conectadas a la interfaz gráfica.

1.2 Alcance

Se conserva la organización del primer proyecto: cada lenguaje tiene su gramática, AST, análisis semántico y generación de C3D. Lat es el punto de entrada para programas que combinan módulos .y y .z; es el único lenguaje con importaciones.

La segunda etapa incorpora herencia simple, encapsulamiento, sobrescritura con @Override, despacho dinámico, this y llamadas a constructores mediante super(...) en Z. Lat permite

declarar referencias con un tipo de clase explícito, almacenar objetos de subclases y utilizar null.

La salida final es un archivo .s. La aplicación permite compilar, consultar las tablas y las cuartetas, ejecutar el ensamblador y proporcionar datos de entrada desde la consola.

1.3 Proceso de compilación

El procesamiento se realiza en el siguiente orden:

1. Lectura del archivo fuente en UTF-8 y selección del lenguaje por su extensión.
2. Tokenización y análisis sintáctico con ANTLR.
3. Construcción del AST correspondiente.
4. Carga de módulos, registro de declaraciones y validación semántica.
5. Consulta de tipos y símbolos anotados para generar C3D.
6. Traducción de las cuartetas finales con RiscVGenerator.
7. Escritura de <nombre>.s en output/, junto al archivo fuente.
8. Ejecución opcional en RARS mediante RarsRunner.

2. Tecnologías utilizadas

Tecnología	**Versión**	**Rol**
Java	17	Implementación del compilador y gestión de procesos
ANTLR	4.13.2	Generación de lexers, parsers y visitors
Maven	3.9.x	Gestión de dependencias y construcción
Lombok	1.18.46	Generación de métodos de acceso
FlatLaf	3.2.1	Apariencia de la interfaz Swing
flatlaf-intellij-themes	3.2.1	Tema Material Darker
Swing	Incluido en el JDK	Editor, explorador, consola y paneles de resultados
RARS	1.6	Ensamblado y simulación del programa RISC-V

Las dependencias Java se encuentran declaradas en pom.xml. El plugin antlr4-maven-plugin, versión 4.13.2, genera visitors con la opción visitor=true. El simulador se distribuye como lib/rars1_6.jar.

El resaltado se implementa con los lexers ANTLR y StyledDocument; no depende de una biblioteca externa de edición o coloreado.

3. Arquitectura general

Capa	**Paquete**	**Responsabilidad**
Presentación	com.navi.ui	Edición, explorador, consola y resultados
Resaltado	com.navi.backend.highlight	Clasificación de tokens para el editor
Orquestación	com.navi.backend.compiler	Compilación completa y ejecución en RARS
AST	com.navi.backend.ast.{lat,y,z}	Representación de cada lenguaje
Análisis generado	com.navi.backend.lexer_parser.{lat,y,z}	Lexers y parsers ANTLR versionados
Semántica	com.navi.backend.semantic	Tipos, símbolos, importaciones y validaciones
C3D	com.navi.backend.c3d	Cuartetas y generación intermedia por lenguaje
Ensamblador	com.navi.backend.c3d.riscv	Análisis y traducción a RISC-V

3.1 Entradas y salidas

CompilerService.compile(Path) acepta .pig, .y y .z. Para integrar varios módulos se compila el .pig que los importa. Compilar un módulo Z aislado permite analizarlo y generar sus miembros, pero no crea por sí mismo una secuencia de uso de sus objetos: el programa principal normalmente se escribe en Lat.

La salida se guarda en <directorio-del-fuente>/output/<nombre>.s. Al generar una nueva salida, el servicio intenta eliminar los archivos regulares de esa carpeta correspondientes a la corrida anterior.

CompilationResult contiene el archivo fuente, el contexto semántico, el texto C3D, la ruta del ensamblador (getRiscVFile()), el estado de éxito y el reporte semántico.

4. Estructura del proyecto

Ruta	**Contenido**
pom.xml	Configuración Maven
docs/	Manuales y anexos
lib/rars1_6.jar	Simulador utilizado para ejecutar
src/main/antlr4/	PigLatin.g4, YLexer.g4, YParser.g4 y Z.g4
src/main/java/com/navi/Main.java	Arranque de la interfaz gráfica
src/main/java/com/navi/backend/ast/	Nodos y visitors del AST
src/main/java/com/navi/backend/lexer_parser/	Analizadores y manejo de indentación
src/main/java/com/navi/backend/semantic/	Modelo semántico, reglas y checkers
src/main/java/com/navi/backend/c3d/	Cuartetas, emisores y traducción RISC-V
src/main/java/com/navi/backend/compiler/	CompilerService, CompilationResult y RarsRunner
src/main/java/com/navi/backend/highlight/	Resaltado de código
src/main/java/com/navi/ui/	Ventana y paneles Swing

4.1 Regeneración de gramáticas

Maven genera analizadores en target/generated-sources/antlr4, en el paquete por defecto. El compilador utiliza las copias versionadas de com.navi.backend.lexer_parser.<lenguaje>. Después de modificar una gramática:

1. Ejecutar mvn generate-sources.
2. Copiar los archivos Java generados del lenguaje a lexer_parser/lat/, lexer_parser/y/ o lexer_parser/z/.
3. Anteponer la declaración package com.navi.backend.lexer_parser.<lenguaje>;.
4. Eliminar los espacios al final de las líneas copiadas.

Se deben conservar los archivos escritos a mano: y/IndentationTokenSource.java y los listeners de las carpetas errors/.

5. Análisis léxico y sintáctico

5.1 Gramáticas

Lenguaje	**Gramática**	**Particularidad**
Lat	PigLatin.g4	Secciones VARIABILES>, MUNERA> y MAIOR>
Y	YLexer.g4 y YParser.g4	Bloques delimitados por indentación
Z	Z.g4	Una clase por archivo, miembros y herencia simple

El Anexo A describe las producciones principales y las ampliaciones. El Anexo C especifica las palabras reservadas y los símbolos.

5.2 Indentación en Y

IndentationTokenSource envuelve al lexer de Y. Compara la indentación de las líneas con una pila y produce los tokens sintéticos INDENT y DEDENT. El parser utiliza estos tokens para reconocer bloques sin llaves.

5.3 Construcción del AST y errores

FileModuleLoader.parseLat, parseY y parseZ preparan los analizadores e instalan CollectingErrorListener. Si existen errores léxicos o sintácticos se interrumpe el procesamiento antes de construir el AST. En caso contrario, un visitor convierte el árbol de parseo en Program, ProgramY o ProgramZ.

Los nodos conservan línea y columna, implementan accept y exponen una etiqueta mediante getNodeLabel(). Los errores recolectados utilizan el formato [línea:columna] mensaje.

5.4 Resaltado en tiempo real

HighlightService reutiliza los tokens de los lexers para clasificarlos como palabras clave, tipos, booleanos, números, cadenas, caracteres, comentarios, operadores, puntuación e identificadores. El editor aplica los estilos sobre su documento Swing.

6. Análisis semántico

6.1 Modelo canónico

Los tres lenguajes comparten semantic.model.Type: VOID, BOOLEAN, INT, DOUBLE, CHAR, STRING, NULL, ERROR, arreglos con tipo y rango, estructuras y clases identificadas por nombre.

AggregateType describe una estructura Y o clase Z: campos ordenados, ámbito de miembros, métodos y constructores. Para las clases también registra la superclase y un identificador de tipo usado durante el despacho dinámico.

SymbolTable organiza un árbol de ámbitos GLOBAL, FUNCTION, CLASS, STRUCT, BLOCK y LOOP. Cada nombre puede asociarse a varios símbolos para admitir sobrecarga. defineUnique registra variables y parámetros; defineCallable registra funciones, métodos y constructores.

Symbol conserva nombre, categoría, tipo, firma, paso por referencia, ámbito, propietario, posición de memoria, ubicación del fuente, modificador de acceso y marca de sobrescritura. posMemory comienza en -1 y se asigna durante la generación intermedia.

TypeCompat centraliza promoción, asignación y comparación. TypeRules valida operadores, condiciones, retornos y accesos. La tabla detallada está en el Anexo B.

6.2 Contexto de análisis

Elemento de SemanticContext	**Propósito**
symbolTable	Símbolos y ámbitos
typeTable	Tipos agregados y relaciones de herencia
loadedModules	Módulos ya cargados
loadedAsts	AST importados que también deben generar C3D
annotations	Tipos resueltos de las expresiones
symbolBindings	Vínculos entre declaraciones y símbolos
errors	Errores semánticos acumulados

annotate, typeOf, bindSymbol y symbolOf comunican la información del análisis a la generación de C3D.

6.3 Declaraciones y cuerpos

El análisis separa el registro de declaraciones de la validación de cuerpos para resolver referencias adelantadas y llamadas recursivas. En Lat se procesan primero las importaciones, después las declaraciones globales y las firmas, y finalmente las funciones y MAIOR>.

Los módulos se cargan y analizan en el orden de sus importaciones. Por ello, en Lat se debe importar la clase padre antes que sus clases hijas. Cada módulo Z registra su clase y sus miembros; TypeTable.resolveInheritance valida la existencia y categoría del padre y rechaza ciclos sobre los tipos registrados hasta ese momento.

En Y se registran los nombres de estructuras, las firmas de funciones y los campos de las estructuras antes de revisar los cuerpos.

6.4 Importaciones

Solo Lat admite import. Las declaraciones importadas se integran por nombre simple en el contexto global; no existen ámbitos de módulo ni acceso cualificado por paquete.

La ruta utiliza puntos y su último segmento es la extensión. Por ejemplo, import z.Persona.z busca z/Persona.z respecto de la carpeta del .pig. Solo se aceptan las extensiones y y z. Y y Z no realizan nuevas importaciones.

6.5 Visitors y checkers

Cada lenguaje dispone de un visitor de declaraciones, un visitor semántico, un comprobador de expresiones, un comprobador de sentencias y un resolvedor de tipos. Los visitors distribuyen el trabajo; los checkers contienen las validaciones y anotan los tipos obtenidos.

6.6 Herencia, encapsulamiento y sobrescritura en Z

La declaración class Hija extends Padre establece herencia simple. La subclase dispone de los miembros heredados de acuerdo con sus permisos de acceso.

Modificador	**Acceso permitido**
public	Desde cualquier clase y desde el programa Lat
private	Únicamente desde la clase que declara el miembro
protected	Desde la clase que declara el miembro y sus subclases
Sin modificador	Igual que private en este proyecto

La sobrescritura exige el mismo nombre y los mismos tipos de parámetros del método heredado. @Override es obligatorio si existe sobrescritura y es un error colocarlo cuando no corresponde a un método heredado. El retorno debe ser compatible con el del padre; se admiten retornos covariantes de clase. La visibilidad del método sobrescrito no puede reducirse.

La sobrecarga se distingue de la sobrescritura: registra firmas diferentes por sus parámetros y se resuelve durante el análisis.

6.7 this, constructores y super

this representa al objeto actual. Permite acceder explícitamente a sus campos y métodos y distinguirlos de parámetros con el mismo nombre. El acceso sin calificar también puede resolverse contra la clase actual.

Si una clase no declara constructores, se registra un constructor público sin argumentos. Si declara alguno, la creación del objeto debe encontrar una firma válida y accesible.

En una subclase, super(argumentos); invoca un constructor del padre y debe ser la primera instrucción del constructor. Si se omite, se requiere un constructor padre sin argumentos accesible, cuya llamada se realiza implícitamente. Si este no existe, el constructor hijo debe invocar explícitamente una firma válida del padre.

6.8 Referencias polimórficas desde Lat

Lat admite un tipo declarado diferente de la clase concreta construida, siempre que sea compatible:

```
esto impresora : novus ImpresoraPDF() : Impresora;
esto pendiente : null : Impresora;
```

El tipo declarado determina qué miembros son válidos durante el análisis. La clase real del objeto determina qué implementación sobrescrita se ejecuta. null es asignable a referencias de clase.

7. Generación de C3D

7.1 Componentes

Clase	**Responsabilidad**
Quad	Representación de cuartetas: operación, resultado y argumentos

Clase	Responsabilidad
C3DEmitter	Temporales, etiquetas, marcos, memoria y emisión
C3DGenerator	Generación de módulos importados y programa de entrada
LatC3DVisitor, YC3DVisitor, ZC3DVisitor	Recorrido por lenguaje
MethodDispatchC3D	Selección dinámica de implementaciones de métodos

Cada visitor se apoya en auxiliares para declaraciones, expresiones, sentencias y resolución de direcciones.

7.2 Stack, heap y arreglos

Cada función, método o constructor utiliza enter <etiqueta>, <tamaño> y leave. Los parámetros y las variables locales se direccionan mediante BP y posiciones en celdas; los globales Lat utilizan GP. Los objetos Z y sus arreglos se almacenan en el heap, administrado mediante HP; las estructuras Y son valores almacenados en el stack.

La primera celda de un objeto Z contiene su identificador de clase. Los campos comienzan a partir de la celda siguiente, considerando el layout heredado. Los arreglos Z incorporan una cabecera con el rango y los tamaños de sus dimensiones, seguida por los elementos.

Los parámetros arreglo de Y ([][]entero matriz) y Lat (series matriz[][] : numerus) se pasan por referencia. Para arreglos multidimensionales se agregan argumentos enteros ocultos con las dimensiones necesarias para calcular índices por filas.

7.3 Booleanos y control de flujo

Las comparaciones y operaciones lógicas producen 0 o 1 mediante etiquetas y saltos condicionales. && y || aplican cortocircuito. Las bifurcaciones utilizan la forma if a op b goto L.

El emisor elimina cuartetas inalcanzables después de una terminación de flujo hasta encontrar una etiqueta alcanzable. La lista final se obtiene mediante finalQuads().

7.4 Llamadas y despacho dinámico

Elemento	Convención de etiqueta
Función	nombre

Elemento	Convención de etiqueta
Método	Propietario_nombre_tiposParametros
Constructor	Propietario_init_tiposParametros

Los métodos y constructores reciben primero el objeto receptor; this ocupa el parámetro cero. El despacho dinámico consulta la etiqueta de clase guardada en el objeto y selecciona la implementación correspondiente entre las subclases conocidas. De esta manera, una referencia de tipo padre puede ejecutar un método sobrescrito por la clase hija.

8. Generación de ensamblador RISC-V

com.navi.backend.c3d.riscv.RiscVGenerator recibe las cuartetas finales, el tamaño de los globales y el contexto semántico. Produce ensamblador para RARS en modo de 32 bits. Las operaciones enteras utilizan instrucciones como add, sub, mul, div y rem; las operaciones decimales emplean instrucciones de precisión simple de la extensión F.

8.1 Componentes del backend

Clase	Responsabilidad
RiscVGenerator	Emisión de datos, instrucciones y rutinas auxiliares
RiscVProgramAnalysis	Análisis de funciones, direcciones, cadenas y tipos necesarios para emitir
RiscVFunction	Información de cada función traducida
RiscVFrameLayout	Cálculo de posiciones y alineación del marco

La identificación de direcciones permite distinguir una suma numérica de un desplazamiento sobre un puntero. Una celda C3D ocupa cuatro bytes y los desplazamientos de memoria se convierten a bytes durante la traducción.

8.2 Estructura del archivo .s

La sección .data reserva _globals, _heap y _HP, además de las cadenas literales y los buffers de lectura necesarios. La sección .text contiene _start, las funciones y las rutinas auxiliares utilizadas.

_start inicializa el puntero del heap, llama a globals_init y a main cuando existen, y termina mediante una llamada al sistema del simulador.

8.3 Marcos de activación

Se usa s0 como frame pointer. El marco guarda ra en -4(fp) y el valor anterior de s0 en -8(fp). Si el bloque de parámetros y locales ocupa N celdas, la celda i se ubica en fp - (8 + 4 × (N - i)); así los arreglos locales crecen hacia direcciones mayores sin invadir el encabezado.

Los temporales se almacenan debajo de ese bloque. El tamaño total del marco se alinea a 16 bytes. El prólogo reserva espacio y guarda los registros; el epílogo los restaura, recupera sp y retorna. Cada invocación tiene sus propios temporales, lo que permite llamadas recursivas.

8.4 Argumentos y retornos

Los primeros ocho argumentos se transfieren en a0 a a7. Los adicionales se colocan en un área alineada del stack. El valor de retorno se entrega en a0. Esta convención interna también contempla el receptor y los argumentos ocultos de dimensiones de arreglos.

8.5 Entrada, salida y cadenas

Las instrucciones ecall utilizan los servicios de RARS. El número del servicio se coloca en a7.

Operación	**Servicio**	**Registro principal**
Imprimir entero	1	a0
Imprimir decimal de precisión simple	2	fa0 (f10)
Imprimir cadena	4	a0
Leer entero	5	Resultado en a0
Leer decimal de precisión simple	6	Resultado en fa0
Leer cadena	8	Buffer en a0, tamaño en a1
Terminar	10	—
Imprimir carácter	11	a0
Leer carácter	12	Resultado en a0

El backend utiliza información de tipos para seleccionar el servicio, incluyendo firmas de funciones y métodos. Cuando se necesita concatenación, incorpora rutinas auxiliares para

convertir valores numéricos y caracteres y construir la cadena resultante. La lectura de cadenas utiliza buffers de 256 bytes y una ayuda que elimina el salto de línea final.

8.6 Representación de decimales y entorno de ejecución

Aunque los lenguajes utilizan los nombres decimalis, flotante y double, el backend los representa como valores flotantes de 32 bits. La precisión efectiva es simple, no la de un double Java de 64 bits.

El archivo generado utiliza los servicios de RARS. No debe ejecutarse como un binario Linux ni asumirse compatible con Venus, cuyas convenciones de entrada y salida son diferentes.

9. Interfaz gráfica

9.1 Ventana principal

CompilerWindow conserva el explorador izquierdo, el editor por pestañas y el panel inferior de resultados. Las pestañas inferiores son **Consola**, **Tabla de símbolos**, **Tabla de tipos** y **C3D**. La barra de estado muestra posición del cursor, archivo y codificación UTF-8.

La barra de herramientas ofrece **Compilar**, **Compilar y ejecutar**, **Detener**, **Guardar** y **Limpiar consola**. El menú Compilar expone las acciones correspondientes; F5 activa Compilar y ejecutar.

9.2 Editor y explorador

EditorPanel utiliza JTextPane, números de línea y UndoManager. Tab inserta cuatro espacios. Se puede deshacer con Ctrl+Z y rehacer con Ctrl+Shift+Z o Ctrl+Y. Las pestañas se cierran con clic central o mediante su menú contextual.

El explorador permite abrir proyectos, abrir archivos con doble clic y crear o eliminar archivos y carpetas. ProjectFileService centraliza las operaciones de archivos y la validación de nombres.

9.3 Compilación desde la interfaz

La compilación guarda el archivo activo y limpia los resultados anteriores. CompilerService se ejecuta en un SwingWorker para mantener disponible la ventana. Si el análisis falla, se presentan los errores en consola. Si termina correctamente, se llenan las tablas y el panel C3D, se informa la ruta del .s y se actualiza el explorador.

9.4 Consola y ejecución

RarsRunner inicia el simulador como un proceso Java independiente, usando el JAR incluido en lib/. La salida estándar y los mensajes de error se conectan a la consola. La lectura se realiza por fragmentos para mostrar también mensajes que no terminan en salto de línea.

ConsolePanel incluye un campo de entrada y el botón **Enviar**. Durante la ejecución, Enter o Enviar remiten una línea al proceso. La aplicación deshabilita las acciones de compilación mientras hay una sesión activa. **Detener** finaliza el proceso; al terminar la ejecución se deshabilita la entrada y se restablecen los controles.

10. Especificación resumida de lenguajes

10.1 Lat — .pig

Lenguaje imperativo de entrada, sensible a mayúsculas. Tiene secciones opcionales de variables globales y funciones, una sección principal MAIOR> y cierre FINIS;. Incluye variables, arreglos, funciones actio y ratio, condicionales si/aliter, ciclos dum/facere/per, perge, interrumpe, reddere, lectura << e impresión >>.

Importa estructuras Y y clases Z. Puede construir objetos con novus, declarar su tipo de referencia explícito y asignar null. Los arreglos se declaran con tipo explícito, por ejemplo series datos[3] : numerus {1, 2, 3};.

10.2 Y? — .y

Lenguaje de indentación significativa. Tiene una sección opcional %estructuras y una sección obligatoria %funciones. No declara variables globales. Incluye estructuras, arreglos, funciones definir, retorno opcional indicado con ->, condicionales, selección múltiple, ciclos y entrada/salida con leer e imprimir.

Los parámetros arreglo se escriben con uno o más pares [] antes del tipo; los parámetros estructura usan {} Nombre variable.

10.3 Zetariano — .z

Lenguaje orientado a objetos con una clase por archivo. Incluye atributos, constructores y métodos sobrecargables, herencia simple con extends, modificadores public/private/protected, @Override, this y super(...).

Conserva creación de objetos y arreglos con new, null, ternario, asignaciones compuestas, condicionales, ciclos, selección múltiple y operaciones print, println y readln. Las clases utilizadas conjuntamente se cargan desde el programa Lat.

11. Compatibilidad de tipos

Los tres lenguajes comparten la promoción numérica CHAR → INT → DOUBLE. La asignación simple permite ensanchamiento; no acepta asignar un decimal a un entero. Las condiciones y los operadores lógicos requieren booleanos.

La suma con un operando cadena tiene resultado cadena. Los operadores aritméticos restantes requieren operandos numéricos; las comparaciones relacionales producen booleanos. La igualdad admite tipos comparables, referencias de clase compatibles por herencia y comparación entre clase y null.

Una instancia de una subclase puede asignarse a una referencia de una superclase. La conversión inversa no se acepta automáticamente. Los arreglos conservan su tipo base y rango; la herencia entre clases no implica covarianza entre arreglos. El Anexo B detalla estos casos.

12. Compilación y ejecución del proyecto

12.1 Requisitos y construcción

Se requieren Java 17, Maven 3.9.x y un entorno gráfico para la aplicación Swing. Desde la raíz del proyecto:

```
mvn compile
```

12.2 Arranque de la interfaz

com.navi.Main abre directamente la ventana. En un IDE se ejecuta esa clase. Desde una terminal Linux:

```
mvn -q compile
CP="target/classes:$(mvn -q dependency:build-classpath -Dmdep.outputFile=/dev/stdout)"
java -cp "$CP" com.navi.Main
```

El punto de entrada actual no recibe un archivo para compilar por argumentos. La selección del fuente se realiza desde la interfaz.

12.3 Uso embebido del servicio

```
CompilerService service = new CompilerService();
```

```
CompilationResult result = service.compile(Path.of("testfile/main.pig"));
if (result.isSuccessful()) {
    String c3d = result.getC3d();
    Path assembly = result.getRiscVFile();
} else {
    System.out.println(result.getMessage());
}
```

12.4 Ejecución externa del ensamblador

Después de compilar testfile/main.pig desde la interfaz:

java -jar lib/rars1_6.jar nc me testfile/output/main.s

RARS lee desde la entrada estándar de la terminal. La acción **Compilar y ejecutar** realiza la integración equivalente dentro de la aplicación.

13. Manejo de errores

Fase	**Mecanismo**	**Efecto**
Lectura del fuente	Validación de archivo y excepciones de E/S	Se informa el fallo
Léxica y sintáctica	CollectingErrorListener	Se interrumpe antes de construir el AST
Semántica	SemanticErrors	Se acumulan errores y se impide generar C3D y nueva salida final
Generación y escritura	Excepciones del servicio	Se informa el error de compilación
Ensamblado y simulación	Salida y código de terminación de RARS	Se muestran mensajes de ejecución en consola

SemanticReporter.report(context) produce el reporte del contexto. Las nuevas validaciones incluyen herencia cíclica, padre inexistente, acceso denegado, sobrescritura sin anotación, anotación incorrecta, retorno incompatible, reducción de visibilidad y llamadas inválidas al constructor padre.